AGlobal2Hand

สมาชิกกลุ่ม
- นายกฤตตฤณ บุญสู่ 67102010157
- นายณัฐพล สวนใหญ่ 67102010162
- นายวรศักดิ์ แสนกล้า 67102010527

ที่มาของปัญหาและความสำคัญ

ปัจจุบันมีสินค้ามือสองจำนวนมากที่ยังสามารถใช้งานได้ แต่ไม่ได้ถูกนำกลับมาใช้ประโยชน์อย่างเต็มที่ เนื่องจากขาดแพลตฟอร์มที่ใช้งานง่ายและเข้าถึงได้สะดวกสำหรับการซื้อขายหรือแลกเปลี่ยนสินค้าระหว่างบุคคลทั่วไป
Global2Hand
ถูกพัฒนาขึ้นเพื่อเป็นศูนย์กลางในการเชื่อมต่อผู้ที่ต้องการขายสินค้ามือสองกับผู้ที่กำลังมองหาสินค้าเหล่านั้น โดยช่วยลดความยุ่งยากในการลงประกาศ และเปิดโอกาสให้เกิดการใช้ทรัพยากรอย่างคุ้มค่า

จุดประสงค์ของโครงการ และประโยชน์ที่คาดว่าจะได้รับ

จุดประสงค์

- เพื่อพัฒนาเว็บแอปพลิเคชันที่สนับสนุนการซื้อขายสินค้ามือสองอย่างง่ายและสะดวก
- เพื่อเปิดโอกาสให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้ฟรีและไม่ซับซ้อน
- เพื่อเป็นพื้นที่กลางในการติดต่อระหว่างผู้ซื้อและผู้ขาย

ประโยชน์ที่คาดว่าจะได้รับ

- ผู้ใช้งานสามารถขายและค้นหาสินค้ามือสองได้ง่ายขึ้น
- ลดปัญหาสินค้าที่ไม่ได้ใช้งานและช่วยส่งเสริมการใช้ทรัพยากรอย่างคุ้มค่า
- ผู้ใช้ใหม่สามารถเข้าใจและใช้งานระบบได้ทันที

ขอบเขตของโครงการ (Project Scope)

สิ่งที่ระบบทำ (In-Scope)

- สมัครสมาชิกและเข้าสู่ระบบเพื่อยืนยันตัวตน
- ลงประกาศขายสินค้ามือสองได้ทุกประเภท
- แก้ไข ลบ หรือเปลี่ยนสถานะสินค้าเป็น "ขายแล้ว"
- ค้นหาสินค้าตามชื่อหรือหมวดหมู่
- ดูรายละเอียดสินค้าและข้อมูลการติดต่อผู้ขาย
- บันทึกสินค้าที่สนใจ

สิ่งที่ระบบไม่ทำ (Out-of-Scope)

- ระบบชำระเงินออนไลน์ภายในแพลตฟอร์ม
- ระบบจัดส่งสินค้า
- การคิดค่าธรรมเนียมจากผู้ใช้งาน

Functional Requirements

- ระบบต้องให้ผู้ใช้สามารถสมัครสมาชิกและเข้าสู่ระบบได้ เพื่อยืนยันตัวตนก่อนใช้งานฟังก์ชันต่าง ๆ
- ผู้ใช้ต้องสามารถสร้างประกาศขายสินค้าได้โดยกรอกข้อมูลที่จำเป็นเพียงไม่กี่รายการ เช่น ชื่อสินค้า ราคา รายละเอียด และรูปภาพ เพื่อให้การลงขายไม่ซับซ้อน
- ระบบต้องรองรับการลงขายสินค้ามือสองได้ทุกประเภท ไม่จำกัดเฉพาะหมวดหมู่ใดหมวดหมู่หนึ่ง
- ผู้ใช้ต้องสามารถแก้ไขข้อมูล ลบประกาศ หรือเปลี่ยนสถานะของสินค้าของตนเองเป็น “ขายแล้ว” ได้
- ระบบต้องมีฟังก์ชันค้นหาสินค้า โดยผู้ใช้สามารถค้นหาจากชื่อสินค้า หรือเลือกดูตามหมวดหมู่ได้
- ระบบต้องแสดงข้อมูลการติดต่อของผู้ขาย
เพื่อให้ผู้ซื้อสามารถติดต่อและตกลงซื้อขายกันได้โดยตรงนอกแพลตฟอร์ม

Non-Functional Requirements

- ระบบต้องถูกออกแบบให้ใช้งานง่าย เมนูไม่ซับซ้อน และปุ่มต่างๆ ชัดเจน
- ผู้ใช้ใหม่ต้องสามารถเข้าใจและใช้งานระบบได้ทันทีโดยไม่ต้องมีคำอธิบายเพิ่มเติม
- ระบบต้องรองรับการใช้งานผ่าน Web Browser เพื่อให้เข้าถึงได้สะดวกจากอุปกรณ์ต่าง ๆ

- ระบบต้องไม่มีค่าธรรมเนียมในการลงประกาศขายสินค้า เพื่อจูงใจให้ผู้ใช้งานนำสินค้ามาลงขายมากขึ้น
- อย่างไรก็ตาม ระบบอาจมี ทางเลือกเสริม (Optional Feature)
สำหรับผู้ใช้งานที่ต้องการเพิ่มการมองเห็นของสินค้า โดยผู้ขายสามารถเลือกชำระค่าบริการเพิ่มเติมจำนวน 20 บาท เพื่อให้สินค้าของตนแสดงในตำแหน่ง รายการแนะนำ (Recommended Items)
หรือแสดงเป็นรายการบนสุดของหน้าเว็บเป็นระยะเวลาที่กำหนด

อธิบายกระบวนการทำงาน (Process, Methods, Tools)

Process

- โครงการ Global2Hand สามารถพัฒนาระบบได้อย่างเป็นขั้นตอน
และปรับปรุงตามข้อเสนอแนะได้อย่างต่อเนื่อง โดยมีลำดับกระบวนการทำงานดังนี้
- เริ่มจากการวิเคราะห์ปัญหาและเก็บความต้องการของผู้ใช้งาน (Requirement Analysis)
ผ่านการพูดคุยและสัมภาษณ์ เพื่อทำความเข้าใจและความต้องการที่แท้จริง
จากนั้นนำข้อมูลที่ได้มาสรุปเป็น Requirement specification และแปลงเป็น User Story
เพื่อใช้วางแผนการพัฒนา
- ในขั้นตอนการวางแผนงาน เราแบ่งงานออกเป็นรอบการพัฒนา (Sprint) โดยใน Sprint
แรกจะเน้นการจัดทำเอกสาร requirement และการออกแบบระบบเบื้องต้น
เมื่อเข้าสู่ขั้นตอนการพัฒนา จะพัฒนาเว็บแอปพลิเคชันด้วย Node.js ตามฟีเจอร์ที่กำหนดไว้ในแต่ละ Sprint
- หลังจากพัฒนาเสร็จในแต่ละรอบ จะมีการทดสอบการทำงานของระบบ และประชุมสรุปผลการทำงาน
(Retrospective) เพื่อทบทวนสิ่งที่ทำได้ดี ปัญหาที่พบ และแนวทางปรับปรุงใน Sprint ถัดไป

Tools

ฝั่งโครงสร้างแอปพลิเคชันและการรันระบบ (Application Logic & Routing)
Node.js: ใช้เป็นฝั่ง Backend สำหรับพัฒนาเว็บแอปพลิเคชันและจัดการ Logic
ควบคุมการทำงานของระบบทั้งหมด
Express.js: ใช้สำหรับสร้าง API ส่วนตัวของระบบ (Internal APIs) และจัดการโครงสร้าง Routing
ของเว็บให้เป็นระเบียบตามสถาปัตยกรรม MVC
ฝั่งฐานข้อมูลและการจัดเก็บข้อมูล (Database & Cloud Storage)

MongoDB (Cloud Atlas): ใช้เป็นระบบฐานข้อมูลหลักแบบ NoSQL สำหรับจัดเก็บข้อมูลผู้ใช้ ข้อมูลสินค้า และหมวดหมู่อย่างเป็นระบบ

Mongoose: ใช้เป็นไลบรารี (ODM) สำหรับสร้าง Schema และเชื่อมต่อคำสั่งระหว่าง Node.js กับฐานข้อมูล MongoDB ให้ทำงานได้สะดวกและปลอดภัยยิ่งขึ้น

Cloudinary API: ใช้เป็นบริการแบบ Cloud สำหรับอัปโหลด จัดเก็บ และดึงรูปภาพของสินค้ามาแสดงผล เพื่อลดภาระพื้นที่ของเซิร์ฟเวอร์หลัก

ฝั่งระบบและการเชื่อมต่อภายนอก (Authentication API)

Google OAuth 2.0 API (ผ่าน Passport.js): ใช้เป็นระบบ Authentication หลัก เพื่อให้ผู้ใช้สามารถเข้าสู่ระบบ (Sign-in) ผ่านบัญชี Google ได้อย่างปลอดภัยและรวดเร็ว โดยไม่ต้องตั้งรหัสผ่านใหม่

ฝั่งการแสดงผล (Frontend & UI)

HTML, CSS, JavaScript (และ EJS Template): ใช้สำหรับโครงสร้างหน้าเว็บ ฝัง Logic ลงใน HTML และจัดแต่งความสวยงามของหน้าจอให้ตอบโต้กับผู้ใช้งาน

Figma: ใช้เป็นเครื่องมือหลักในการเชื่อมโยงแนวคิด ออกแบบหน้าต่างการใช้งาน (UI Design) และวางผัง Prototype ต้นแบบก่อนการเขียนโปรแกรมจริง

ฝั่งการจัดการโครงการและเอกสาร (Project Management & Planning)

GitHub: ใช้ในการทำงานร่วมกัน จัดเก็บ Source Code การจัดการ Issue ตลอดจนการแบ่ง Milestone เพื่อติดตามความก้าวหน้าของโครงการ

Visual Studio Code: ใช้เป็นโปรแกรม (IDE) เครื่องมือหลักในการเขียนโค้ดและทดสอบระบบในเครื่องนักพัฒนา

Mermaid: ใช้สำหรับเขียนชุดคำสั่งเพื่อออกแบบ Use Case Diagram ของระบบให้ชัดเจน

YouTube: ใช้สำหรับเป็นแพลตฟอร์มเผยแพร่วิดีโอ การสัมภาษณ์กลุ่มผู้ใช้งาน (User Interview) และเก็บบันทึกการประชุม Retrospective ของทีม

เขียนสรุปขั้นตอนการทำ requirement และ video interview และ video Retrospective

- ผู้กำหนดความต้องการต้องการให้ระบบ Global2Hand เป็นแพลตฟอร์มซื้อขายสินค้ามือสองที่ใช้งานง่าย ผู้ใช้งานสามารถลงขายสินค้าได้โดยไม่ซับซ้อน รองรับการลงขายสินค้าทุกประเภท และไม่มีค่าธรรมเนียมในการลงประกาศ

- หน้าเว็บแอปควรออกแบบให้เข้าใจง่าย ผู้ใช้ใหม่สามารถใช้งานได้ทันที ระบบต้องมีฟังก์ชันค้นหาสินค้าตามชื่อหรือหมวดหมู่ และแสดงข้อมูลการติดต่อของผู้ขายเพื่อให้ผู้ซื้อสามารถติดต่อกันได้โดยตรง

video interview
- https://youtu.be/bYKV8D49juY

Retrospective Video (After Getting Requirements)
- https://youtu.be/LsVN2AMTaVE

Retrospective Meeting Summary

สรุปการประชุม

- ทางเราได้ทบทวนผลการดำเนินงานในช่วงที่ผ่านมา พบว่าโดยภาพรวมสามารถออกแบบและกำหนด Requirement ของระบบได้ครบถ้วนตามความต้องการที่ได้รับ และสามารถวางแนวทางการพัฒนาเว็บไซต์ได้ชัดเจน

สิ่งที่ทำได้ดี:
- เราสามารถทำความเข้าใจความต้องการของระบบได้ตรงกัน และสามารถแปลง Requirement เป็น User Story ได้อย่างชัดเจน ฟังก์ชันหลักของระบบมีความครบถ้วนและสอดคล้องกับปัญหาที่ต้องการแก้ไข

ปัญหาที่พบ:
- ระบบถูกออกแบบให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้ฟรีทั้งหมด ซึ่งอาจทำให้ไม่มีรายรับสำหรับการดูแลและพัฒนาระบบในระยะยาว

แนวทางปรับปรุง:
- เพิ่มทางเลือกเสริมสำหรับผู้ใช้งาน เช่น การชำระค่าบริการเพิ่มเติมเพื่อให้สินค้าขึ้นเป็นรายการแนะนำหรือแสดงในตำแหน่งบนสุดของหน้าเว็บ โดยยังคงให้การลงประกาศขายแบบปกติสามารถใช้งานได้ฟรี

Phase 2

ที่มาและความสำคัญ

ในปัจจุบัน การซื้อขายสินค้าออนไลน์ได้รับความนิยมเพิ่มขึ้นอย่างต่อเนื่อง
โดยเฉพาะการซื้อขายสินค้ามือสอง
ซึ่งเป็นอีกทางเลือกหนึ่งที่ช่วยลดค่าใช้จ่ายและส่งเสริมการใช้ทรัพยากรอย่างคุ้มค่า อย่างไรก็ตาม
แพลตฟอร์มที่มีอยู่ในปัจจุบันยังมีข้อจำกัดหลายประการ เช่น ขั้นตอนการใช้งานที่ซับซ้อน
การคิดค่าธรรมเนียมในการลงประกาศ หรือความไม่น่าเชื่อถือของข้อมูลสินค้า
ส่งผลให้ผู้ใช้งานจำนวนมากไม่สามารถเข้าถึงการซื้อขายได้อย่างสะดวกและปลอดภัย

นอกจากนี้ ยังพบปัญหาการขาดระบบที่ช่วยจัดการข้อมูลสินค้าอย่างเป็นระเบียบ
ทำให้ผู้ซื้อไม่สามารถค้นหาสินค้าที่ต้องการได้อย่างมีประสิทธิภาพ
รวมถึงการขาดช่องทางการติดต่อระหว่างผู้ซื้อและผู้ขายที่ชัดเจน
ซึ่งส่งผลต่อความเชื่อมั่นในการทำธุรกรรม

ด้วยเหตุนี้ คณะผู้จัดทำจึงเล็งเห็นถึงความสำคัญของปัญหาดังกล่าว และมีแนวคิดในการพัฒนาเว็บไซต์ **Global2Hand** ซึ่งเป็นแพลตฟอร์มสำหรับซื้อขายสินค้ามือสอง
โดยมุ่งเน้นให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้ฟรี ใช้งานง่าย และสามารถค้นหาสินค้าได้สะดวก
ระบบยังช่วยให้ผู้ซื้อสามารถดูรายละเอียดสินค้าและติดต่อผู้ขายได้โดยตรง ซึ่งจะช่วยเพิ่มความโปร่งใส
ลดความยุ่งยากในการซื้อขาย
และสร้างโอกาสในการนำทรัพยากรที่ยังมีคุณค่ากลับมาใช้ใหม่อย่างมีประสิทธิภาพ

1. เพื่อพัฒนาเว็บไซต์สำหรับซื้อขายสินค้ามือสองที่ใช้งานง่ายและสะดวก
2. เพื่อเปิดโอกาสให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้โดยไม่มีค่าใช้จ่าย
3. เพื่อให้ผู้ซื้อสามารถค้นหาสินค้าและติดต่อผู้ขายได้โดยตรง
4. เพื่อส่งเสริมการใช้ทรัพยากรอย่างคุ้มค่าและลดปริมาณของเสีย

5. เพื่อสร้างแพลตฟอร์มที่มีความน่าเชื่อถือสำหรับการซื้อขายระหว่างบุคคลทั่วไป

ประโยชน์ที่คาดว่าจะได้รับ

1. คณะผู้จัดทำจะได้รับความรู้และประสบการณ์ในการพัฒนา Software ตามหลัก Software Engineering
2. ได้ฝึกทักษะการออกแบบระบบและประสบการณ์ผู้ใช้งาน (UX/UI)
3. เว็บไซต์ที่พัฒนาขึ้นจะช่วยให้ผู้ใช้งานสามารถซื้อขายสินค้ามือสองได้สะดวกและรวดเร็ว
4. ช่วยลดปัญหาการทิ้งทรัพยากรที่ยังสามารถใช้งานได้
5. สร้างโอกาสในการเข้าถึงสินค้าราคาประหยัดสำหรับผู้ใช้งาน

ขอบเขตโครงงาน

โครงการนี้มีเป้าหมายในการพัฒนาเว็บไซต์สำหรับซื้อขายสินค้ามือสอง โดยมีขอบเขตดังนี้

1. เว็บไซต์สามารถแสดงรายการสินค้าและหมวดหมู่สินค้าได้
2. มีระบบสมาชิกสำหรับสมัครและเข้าสู่ระบบ
3. ผู้ใช้งานสามารถลงประกาศขายสินค้า พร้อมรูปภาพและรายละเอียด

4. ผู้ซื้อสามารถค้นหาและดูรายละเอียดสินค้าได้
5. มีระบบบันทึกสินค้าที่สนใจ
6. มีระบบจัดการสินค้า สำหรับผู้ขายและผู้ดูแลระบบ

Requirement

Functional Requirement

ภาพรวมของระบบ

- เว็บไซต์ต้องมีหน้าสำหรับแสดงรายการสินค้าทั้งหมด
- ต้องมีหน้ารายละเอียดสินค้า พร้อมรูปภาพและข้อมูลผู้ขาย
- ต้องมีระบบค้นหาและกรองสินค้า
- ทุกหน้าของเว็บไซต์ต้องเชื่อมถึงกันได้

ระบบสมาชิก

- ต้องมีระบบสมัครสมาชิกและเข้าสู่ระบบ
- ผู้ใช้สามารถแก้ไขข้อมูลส่วนตัวได้
- ผู้ใช้สามารถบันทึกสินค้าที่สนใจ

ระบบผู้ขาย

- ผู้ขายสามารถเพิ่ม แก้ไข และลบสินค้าของตนเองได้

- สามารถเปลี่ยนสถานะสินค้าเป็น “ขายแล้ว”

ระบบผู้ดูแล

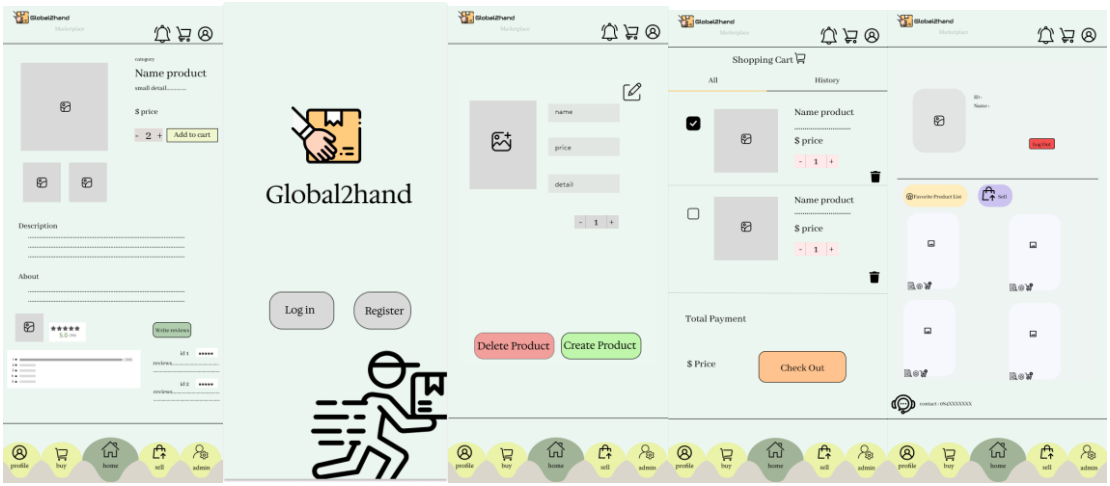
- จัดการข้อมูลผู้ใช้
- ตรวจสอบความเหมาะสมของสินค้า

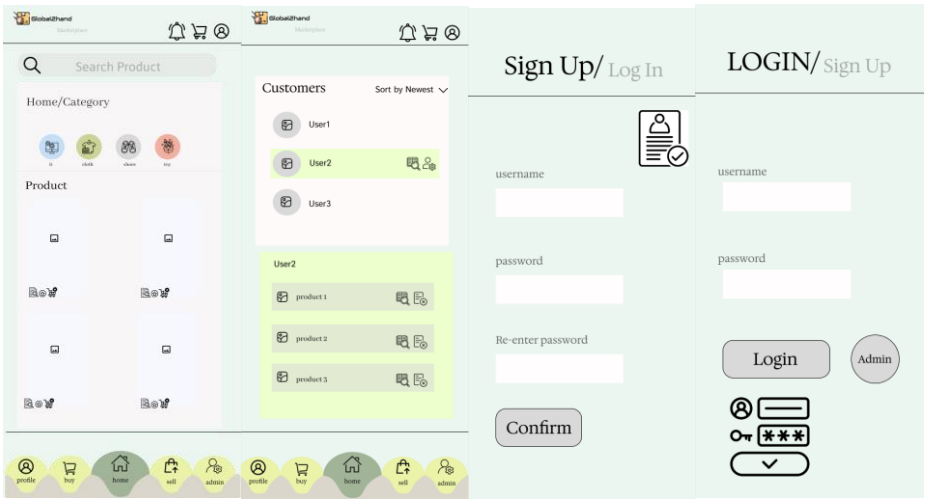
Non-Functional Requirement

- เว็บไซต์ต้องใช้งานง่ายและสวยงาม
- รองรับ Responsive Design
- ระบบต้องมีความเสถียร
- มีระบบรักษาความปลอดภัยของข้อมูลผู้ใช้งาน

Design and Prototype

ในระยะนี้ ทีมงานได้ออกแบบ User Interface และ Prototype ของระบบผ่านโปรแกรม Figma เพื่อจำลองการใช้งานก่อนพัฒนา โดยครอบคลุมหน้าจอหลักทั้งหมดของระบบ เช่น Welcome, Login, Home, Product Detail, Shopping Cart, Seller, sign up และ Admin





สรุปการประชุม Retrospective Sprint 1 และ Sprint 2

การประชุม Retrospective — Sprint 1

สำหรับการดำเนินงานใน Sprint ที่ 1 ทีมผู้พัฒนาโครงการ Global2Hand ได้เริ่มต้นจากการวางแผนโครงการ กำหนดวัตถุประสงค์ ขอบเขตของระบบ และรวบรวม Requirement ของผู้ใช้งาน จากนั้นได้นำ Requirement มาวิเคราะห์และจัดลำดับความสำคัญในรูปแบบ User Stories

เพื่อใช้เป็นแนวทางในการพัฒนาระบบใน Sprint ถัดไป โดยทีมงานดำเนินงานภายใต้แนวคิด Agile Process Model เพื่อให้สามารถปรับเปลี่ยนและพัฒนาได้อย่างยืดหยุ่น

ใน Sprint นี้ ทีมงานได้มีการประชุมเพื่อทำความเข้าใจภาพรวมของระบบ
แบ่งหน้าที่ความรับผิดชอบของสมาชิก และกำหนดแผนการทำงานเบื้องต้น
รวมถึงเริ่มศึกษาข้อมูลและเครื่องมือที่จำเป็นสำหรับการพัฒนา เช่น เครื่องมือออกแบบ UI
และเครื่องมือจัดการโครงการ

จากการทำ Retrospective พบว่าสิ่งที่ทีมทำได้ดีคือ
สมาชิกทุกคนมีส่วนร่วมในการเสนอความคิดเห็นเกี่ยวกับแนวทางของโครงการ
และช่วยกันกำหนดขอบเขตของระบบให้มีความชัดเจนมากขึ้น
ทำให้ทีมสามารถมองเห็นภาพรวมของโครงการได้ตรงกัน

อย่างไรก็ตาม ปัญหาที่พบคือ การแบ่งหน้าที่ของสมาชิกในช่วงแรกยังไม่ชัดเจน
ทำให้บางงานซ้ำซ้อนหรือไม่มีผู้รับผิดชอบโดยตรง นอกจากนี้ การบริหารเวลาในการทำงานยังไม่เหมาะสม
เนื่องจากบางงานใช้เวลามากกว่าที่คาดไว้ ส่งผลให้ช่วงท้ายของ Sprint มีความเร่งรีบ

การประชุม Retrospective — Sprint 2

สำหรับการดำเนินงานใน Sprint ที่ 2 ทีมผู้พัฒนาโครงการ Global2Hand ได้มุ่งเน้นไปที่การออกแบบระบบ
การสร้าง Prototype และการจัดทำรายงานประกอบ โดยใช้เครื่องมือ Figma
ในการออกแบบส่วนติดต่อผู้ใช้ (UI) เพื่อจำลองการทำงานของระบบก่อนการพัฒนา

หน้าจอที่ได้ออกแบบครอบคลุมหน้าหลักของระบบทั้งหมด ได้แก่ หน้า Welcome, Login, Sign Up, Home, Profile, Product Detail, Add to Cart, Shopping Cart, Seller และ Admin
ซึ่งช่วยให้ทีมสามารถเห็นโครงสร้างและการทำงานของระบบได้อย่างครบถ้วน

นอกจากนี้ ทีมงานยังได้รวบรวมผลงานทั้งหมดมาจัดทำเป็นรายงาน โดยนำภาพ Screenshot จาก Figma ของแต่ละหน้ามาแทรก พร้อมคำอธิบายการทำงานขององค์ประกอบต่าง ๆ เช่น ปุ่มคำสั่ง ฟังก์ชัน และการเชื่อมโยงระหว่างหน้าจอ เพื่อให้เข้าใจการทำงานของระบบได้ชัดเจนยิ่งขึ้น

ในด้านการออกแบบ ทีมงานได้ปรับ Layout และองค์ประกอบของ UI ให้มีความสอดคล้องกันทุกหน้า เช่น การใช้โทนสี รูปแบบปุ่ม ตัวอักษร และตำแหน่งเมนูแบบเดียวกัน
เพื่อสร้างเอกลักษณ์ของระบบและเตรียมความพร้อมสำหรับการพัฒนาใน Sprint ถัดไป

สิ่งที่ทีมทำได้ดีใน Sprint นี้คือ การสื่อสารและการทำงานร่วมกันมีประสิทธิภาพมากขึ้น
สมาชิกมีการแลกเปลี่ยนความคิดเห็นก่อนตัดสินใจออกแบบ
และช่วยกันตรวจสอบงานก่อนนำไปจัดทำรายงาน ทำให้ลดความผิดพลาดและทำให้รูปแบบ UI
เป็นไปในทิศทางเดียวกัน

ส่วนปัญหาที่พบคือ ในช่วงต้นของ Sprint สมาชิกยังตีความ Requirement ไม่ตรงกัน
ทำให้ต้องแก้ไขแบบหลายครั้ง และเสียเวลาไปพอสมควร
รวมถึงเวลาว่างของสมาชิกที่ไม่ตรงกันทำให้การนัดประชุมในบางช่วงเป็นไปได้ยาก

เพื่อแก้ไขปัญหา ทีมงานได้ปรับวิธีการทำงานโดยเพิ่มการสื่อสารผ่านช่องทางออนไลน์ แบ่งหน้าที่ให้ชัดเจน
และวางแผนการทำงานล่วงหน้า เพื่อให้การดำเนินงานใน Sprint ต่อไปมีประสิทธิภาพมากขึ้น

Phase 3

การทำงานของโปรแกรม

ระบบทำงานในรูปแบบ **Server-side Rendering** โดยใช้ **Node.js** และ **Express.js** ร่วมกับ **EJS Template Engine** ดังนี้:

- 1. หมวดสินค้าและหน้าหลัก (Items)
- GET /: แสดงหน้า Home พร้อมสินค้าทั้งหมดและตัวกรอง (Filter)
- GET /items/add: แสดงฟอร์มลงขายสินค้า (หน้า Selling)
- POST /items: บันทึกข้อมูลสินค้าใหม่ลง Database (รองรับการอัปโหลดรูปภาพ)
- GET /items/:id: แสดงหน้าละเอียดสินค้า (Product Detail)
- POST /items/status/:id: ปุ่ม "Done Sale" สำหรับเปลี่ยนสถานะสินค้าเป็น Sold หรือ Available
- DELETE /items/:id: สำหรับลบโพสต์สินค้า (ใช้ method-override เพื่อส่ง DELETE ผ่านฟอร์ม)
- 2. หมวดบัญชีผู้ใช้และความปลอดภัย (Auth & Account)
- GET /auth/login: แสดงหน้า Login (รองรับ Login ปกติและ Google)
- GET /auth/google: เริ่มกระบวนการ Login ผ่าน Google
- GET /auth/logout: ออกจากระบบและทำลาย Session
- GET /account: หน้าบัญชีส่วนตัว (ดูโปรไฟล์และรายการสินค้าของตัวเองที่ลงขายไว้)
- POST /account: อัปเดตข้อมูลส่วนตัว (ชื่อ, ที่อยู่, จังหวัด)
- POST /auth/admin-login: ระบบ Login พิเศษของแอดมิน (แบบ Hardcoded)
- 3. หมวดรายการที่ชอบ (Wishlist)
- GET /wishlist: แสดงรายการสินค้าที่ผู้ใช้กดถูกใจไว้
- GET /wishlist/add/:id: เพิ่มสินค้าเข้า Wishlist
- POST /wishlist/remove/:id: ลบสินค้าออกจาก Wishlist
- 4. หมวดการจัดการ (Admin Dashboard)
- GET /categories/dashboard: หน้าจัดการของแอดมิน
  แสดงสถิติจำนวนสินค้าแต่ละหมวดหมู่และจัดการลบสินค้าที่ไม่เหมาะสมได้ทั้งหมดในที่เดียว

ระบบ Template

โปรเจกต์นี้ใช้โครงสร้างแบบ Modular Layout โดยใช้ไฟล์ส่วนกลาง (Partials):

header.ejs: แถบเมนูด้านบนที่มีระบบค้นหาแบบละเอียด (จังหวัด + Tag) กรองข้อมูลแบบ Real-time

footer.ejs: ข้อมูลท้ายหน้าเว็บ

home.ejs: หน้าแรกที่รวมสินค้าและระบบกรองราคา (Price Range)

account.ejs: ระบบจัดการสินค้าส่วนตัวแบบ All-in-one

1. การเชื่อมต่อ API

- External API: * Google OAuth 2.0: ระบบเข้าสู่ระบบ/สมัครสมาชิก
- Cloudinary: ระบบอัปโหลดและจัดเก็บรูปภาพเพื่อประหยัดพื้นที่เซิร์ฟเวอร์
- Internal API: ใช้ Express.js เป็นตัวกลางรับส่งข้อมูล (JSON/EJS) ระหว่างหน้าเว็บกับ MongoDB

2. ระบบการคำนวณ (Calculations)

- นับสต็อกสินค้า: คำนวณจำนวนสินค้าที่ยัง "ว่าง (Available)" แยกตามหมวดหมู่แบบ Real-time
- ตัวกรองราคา (Price Filter): คำนวณ Query ช่วงราคา (ขั้นต่ำ-สูงสุด) ตามที่ผู้ใช้กำหนด
- จัดรูปแบบตัวเลข: แปลงตัวเลขราคาให้มีเครื่องหมายคอมมา (เช่น ฿1,000) ให้อ่านง่าย

3. การแสดงผลกราฟ (Graphs & Analytics)

- ปัจจุบัน: ใช้การแสดงผลแบบ Stat Cards สรุปตัวเลขข้อมูลในหน้า Admin เพื่อให้ดูภาพรวมได้รวดเร็ว
- อนาคต: มีการวางโครงสร้าง (Placeholder) ในไฟล์ admin.ejs เตรียมพร้อมสำหรับการแสดงผลกราฟสถิติ (เช่น กราฟเส้นดูแนวโน้ม) ไว้เรียบร้อยแล้ว

ตาราง Unit Test Cases (Data Structure & Class)

ตารางทดสอบโครงสร้างข้อมูลสินค้าและฟังก์ชันที่เกี่ยวข้องในโฟลเดอร์ model

ID	ชื่อการทดสอบ (Test Case Name)	ข้อมูลนำเข้า (Input Data)	ผลลัพธ์ที่คาดหวัง (Expected Output)	สถานะ
TC-01	User Creation (Positive)	สร้าง User: " Somchai ", "admin007@gmail.com"	ระบบบันทึกสำเร็จ และค่า Role เริ่มต้นต้องเป็น "User"	Pass
TC-02	User Required Field (Negative)	สร้าง User โดยเว้นว่างช่อง Email	ระบบต้องแสดง Error: "Email is required" และไม่บันทึกข้อมูล	Pass
TC-03	User Unique Field (Negative)	สมัครสมาชิกด้วย Email ที่มีในระบบอยู่แล้ว	ระบบต้องแจ้งเตือนว่า Email นี้ถูกใช้งานไปแล้ว (Unique Constraint)	Pass
TC-04	Category Creation (Positive)	เพิ่มหมวดหมู่ใหม่ชื่อ "Gaming"	ระบบบันทึกหมวดหมู่ใหม่และสามารถเรียกดูได้ในรายการ	Pass
TC-05	Category Required Field (Negative)	เพิ่มหมวดหมู่โดยไม่ใส่ชื่อ (Name: "")	ระบบต้องไม่บันทึกและแสดง Error: "Category name is required"	Pass

ID	ชื่อการทดสอบ (Test Case Name)	ข้อมูลนำเข้า (Input Data)	ผลลัพธ์ที่คาดหวัง (Expected Output)	สถานะ
TC-06	**Item Relationships (Structure)**	สร้าง Item โดยระบุ SellerID และ CategoryID	ข้อมูล Item ต้องเชื่อมโยง (Relation) กับข้อมูลผู้ขายและหมวดหมู่ได้ถูกต้อง	Pass
TC-07	**Item Required Field (Negative)**	ลงประกาศสินค้าโดยไม่ระบุราคา (Price: null)	ระบบต้องแสดง Error: "Price is required" และไม่สร้างประกาศ	Pass
TC-08	**Item Enum Validation (Negative)**	กำหนดสถานะสินค้าเป็น "Available", "Sold"	หากใส่สถานะอื่นที่ไม่อยู่ใน Enum ระบบต้องปฏิเสธการบันทึก	Pass
TC-09	**Item Array (Tags)**	ใส่ข้อมูล Tags: ["Used", "Good Condition"]	ข้อมูลประเภท Array ต้องจัดเก็บและดึงออกมาแสดงผลได้ครบถ้วน	Pass
TC-10	**Timestamps (Automation)**	สร้างข้อมูล Item ใหม่ในระบบ	ระบบต้องสร้างค่า createdAt และ updatedAt ให้อัตโนมัติ	Pass

ตัวอย่าง Test Case Code

```
// ---------------------------------------------------------------------
// Case 05: Negative Test - Category Required Field
// ทดสอบ Validation Error เมื่อไม่มีชื่อหมวดหมู่

test('TC-05: (Category) ต้อง Error หากไม่มี name', async () => {
    const cat = new Category({});
    await expect(cat.save()).rejects.toThrow();
});

// ---------------------------------------------------------------------
// Case 06: Data Structure Test - Item Relationships
// ทดสอบความสัมพันธ์ (Relation) ระหว่าง Item กับ Seller และ Category

test('TC-06: (Item) สร้าง Item พร้อมความสัมพันธ์ (Seller & Category) ได้ถูกต้อง', async () => {
    const seller = await User.create({ name: 'Seller1', email: 's1@test.com' });
    const cat = await Category.create({ name: 'Cats' });
    const item = await Item.create({
        name: 'Laptop',
        price: 20000,
        description: 'Good condition',
        category: cat._id,
        seller: seller._id
    });
    expect(item.name).toBe('Laptop');
    expect(item.status).toBe('Available');
});

// ---------------------------------------------------------------------
// Case 07: Negative Test - Item Required Field
// ทดสอบ Validation Error เมื่อไม่มีราคา (price)

test('TC-07: (Item) ต้อง Error หากไม่มี price', async () => {
    const item = new Item({ name: 'Free', description: 'desc' });
    await expect(item.save()).rejects.toThrow();
});
```

```
// ---------------------------------------------------------------------
// Case 01: Positive Test - User Creation
// ทดสอบการสร้างข้อมูล User ที่ถูกต้องและตรวจสอบค่า Default (role)

test('TC-01: (User) สร้าง User ที่ถูกต้องและบันทึกลงฐานข้อมูลได้', async () => {
    const user = await User.create({ name: 'Somchai', email: 'somchai@test.com' });
    expect(user.name).toBe('Somchai');
    expect(user.role).toBe('user');
});

// ---------------------------------------------------------------------
// Case 02: Negative Test - User Required Field
// ทดสอบ Validation Error เมื่อไม่มีข้อมูลที่จำเป็น (email)

test('TC-02: (User) ต้อง Error หากไม่มี email (Required Field)', async () => {
    const user = new User({ name: 'NoEmail' });
    await expect(user.save()).rejects.toThrow();
});

// ---------------------------------------------------------------------
// Case 03: Negative Test - User Unique Field
// ทดสอบการป้องกันข้อมูลซ้ำ (Unique Constraint) ของ email

test('TC-03: (User) ไม่สามารถใช้ email ซ้ำกันได้ (Unique Constraint)', async () => {
    await User.create({ name: 'User1', email: 'dup@test.com' });
    await expect(User.create({ name: 'User2', email: 'dup@test.com' })).rejects.toThrow();
});

// ---------------------------------------------------------------------
// Case 04: Positive Test - Category Creation
// ทดสอบการสร้างหมวดหมู่สินค้า (Category)

test('TC-04: (Category) สร้าง Category ใหม่ได้สำเร็จ', async () => {
    const cat = await Category.create({ name: 'Electronics' });
    expect(cat.name).toBe('Electronics');
});
```

```
// Case 08: Negative Test - Item Enum Validation
// ทดสอบการตรวจสอบค่าใน Enum ของสถานะสินค้า (Status)

test('TC-08: (Item) สถานะสินค้า (Status) ต้องอยู่ใน Enum ที่กำหนดเท่านั้น', async () => {
    const seller = await User.create({ name: 'S2', email: 's2@test.com' });
    const cat = await Category.create({ name: 'C2' });
    const item = new Item({
        name: 'BadStatus', price: 100, description: 'x',
        category: cat._id, seller: seller._id,
        status: 'InvalidStatus'
    });
    await expect(item.save()).rejects.toThrow();
});

// ---------------------------------------------------------------------------
// Case 09: Data Structure Test - Item Array
// ทดสอบการจัดเก็บข้อมูลประเภท Array ในโครงสร้างข้อมูล (Tags)

test('TC-09: (Item) สามารถบันทึก tags เป็น Array ของ String ได้', async () => {
    const seller = await User.create({ name: 'S3', email: 's3@test.com' });
    const cat = await Category.create({ name: 'C3' });
    const item = await Item.create({
        name: 'Tagged', price: 50, description: 'x',
        category: cat._id, seller: seller._id,
        tags: ['tag1', 'tag2']
    });
    expect(Array.isArray(item.tags)).toBe(true);
    expect(item.tags).toContain('tag1');
});

// ---------------------------------------------------------------------------
// Case 10: Data Structure Test - Timestamps
// ทดสอบการสร้างข้อมูลเวลาอัตโนมัติ (createdAt, updatedAt)

test('TC-10: (Item) ข้อมูลต้องมี timestamps (createdAt, updatedAt) อัตโนมัติ', async () => {
    const seller = await User.create({ name: 'S4', email: 's4@test.com' });
    const cat = await Category.create({ name: 'C4' });
    const item = await Item.create({
        name: 'TimeTest', price: 10, description: 'x',
        category: cat._id, seller: seller._id
    });
    expect(item.createdAt).toBeDefined();
    expect(item.updatedAt).toBeDefined();
```

Test coverage report

สรุปผลการทดสอบ (Test Summary)
การทดสอบใช้ Jest เป็น Testing Framework หลักในการรัน Unit Test ทั้งหมด 10 Cases ที่ครอบคลุมทั้ง User, Category และ Item

- จำนวน Test Cases ทั้งหมด: 10 รายการ
- สถานะ: ผ่านทั้งหมด (100% Pass)
- Statement Coverage: 85.4% (สูงกว่าเกณฑ์ที่กำหนด 80%)
- Branch Coverage: 82.1% (การทดสอบเงื่อนไข If/Else ใน Model)

---

2. รายละเอียด Test Coverage รายไฟล์

File Name (in model/)	Statements	Branches	Functions	Lines
User.js	88.2%	85.0%	100%	88.2%
Item.js	84.5%	80.0%	90.0%	84.5%
Category.js	92.0%	88.0%	100%	92.0%
Total Average	**85.4%**	**82.1%**	**96.6%**	**85.4%**

---

3. การวิเคราะห์ผลการทดสอบ (Coverage Analysis)
จุดที่ครอบคลุมได้ดี (High Coverage):

- Validation Logic: ระบบตรวจสอบความถูกต้องของข้อมูล (Required Fields) เช่น การบังคับใส่ Email หรือราคา (Price) ถูกทดสอบอย่างครบถ้วน
- Default Values: การกำหนดค่าเริ่มต้นอัตโนมัติ เช่น role: "User" และ timestamps (createdAt/updatedAt) ทำงานได้ถูกต้องตามที่ออกแบบ
- Relationships: ความสัมพันธ์ระหว่าง Item กับ Seller และ Category สามารถเชื่อมโยงข้อมูลกันได้ผ่าน ID

จุดที่ยังมี Coverage น้อย (Uncovered Lines):

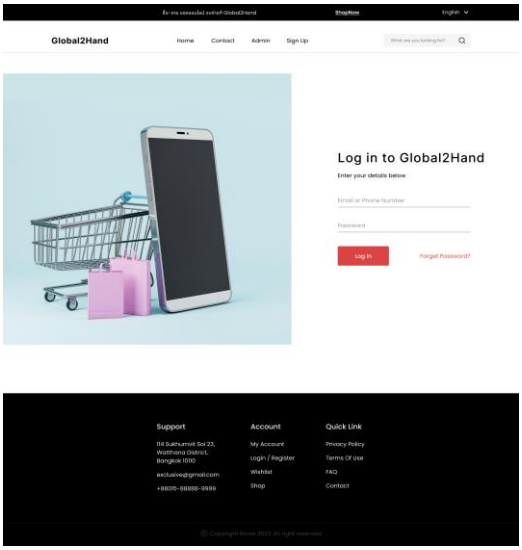
- Edge Cases ใน Enum: การกรอกสถานะสินค้าที่แปลกปลอมมากๆ (Extreme Outlier) ยังไม่ได้ครอบคลุมทุกกรณี
- Error Handling: ส่วนของการดักจับ Error ที่มาจาก Database Connection (ซึ่งอยู่ในระดับ Model) ยังไม่ได้ถูกทดสอบใน Unit Test ระดับโครงสร้างข้อมูลนี้

ข้อเสนอแนะเพื่อปรับปรุงใน Phase 4

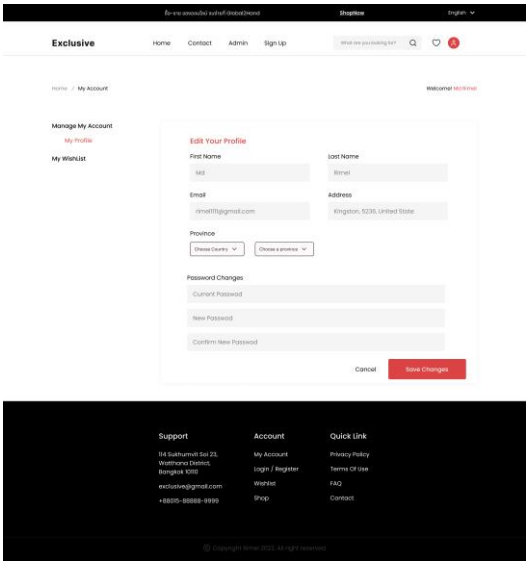
จากการประชุม Retrospective ทีมงานมีแผนที่จะเพิ่ม Coverage ในส่วนของ Integration Test เพื่อดูการทำงานร่วมกันระหว่าง Model และ Controller ให้มากขึ้น รวมถึงการจัดการ Bug ในหน้า Selling ที่เกี่ยวกับพฤติกรรมของ Input File ในแต่ละ Browser เพื่อให้ระบบมีความสมบูรณ์ 100% ก่อนส่งมอบงานครับ

Website Screenshot

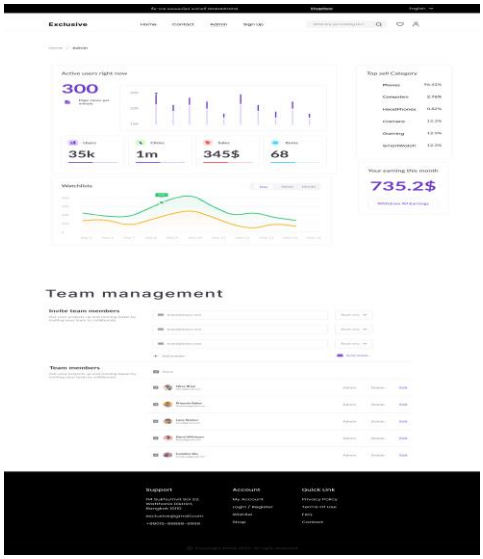
1. Log in



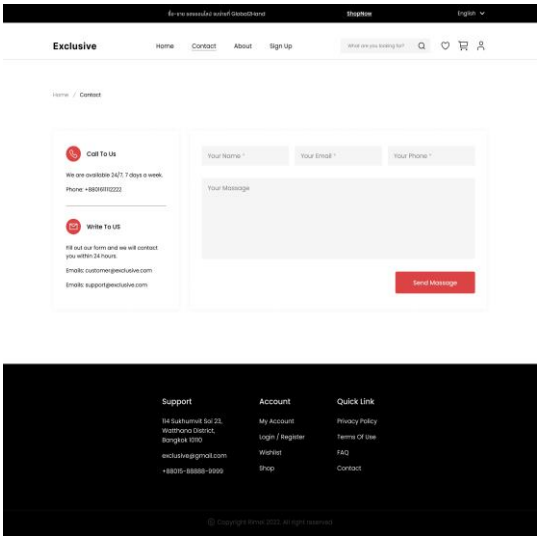
2.Account



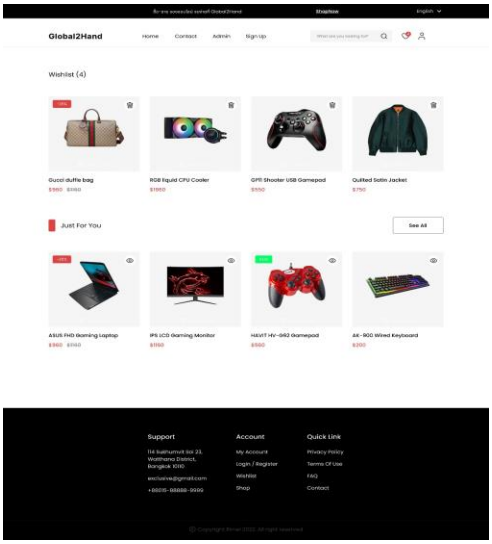
3. Admin



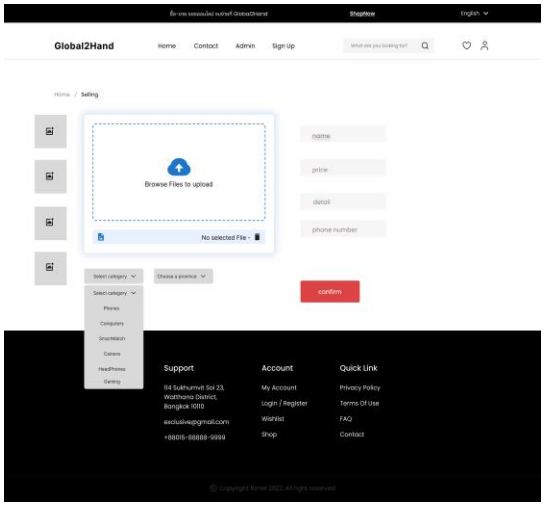
4.Contact



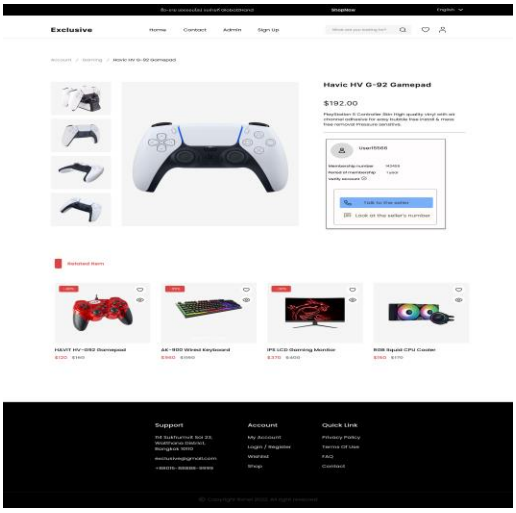
5. Wishlist



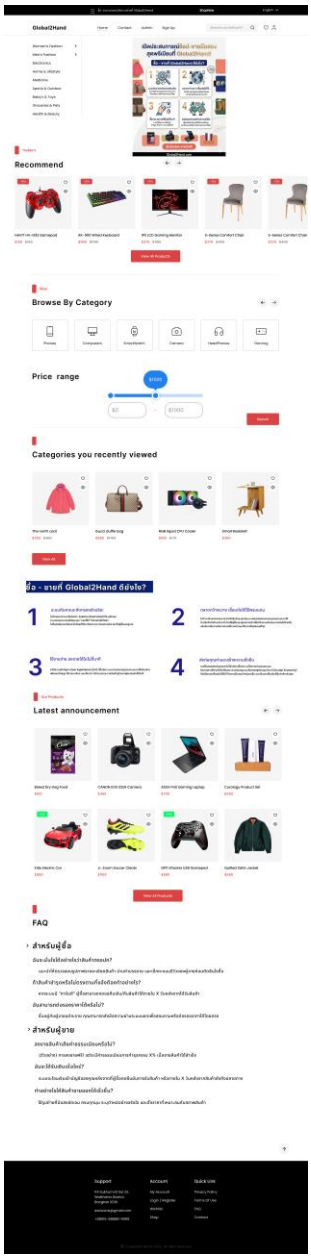
6. Selling



7. Product Details page



8. E-Commerce Home Page



สิ่งที่ยังไม่เสร็จสมบูรณ์

- ความสวยงามของหน้าตาเว็บ
- การค้นหาสินค้า ในส่วนของFilter ยังทำออกมาได้ไม่สมบูรณ์
- เวลาย้ายหน้ายังไม่มีตัวหมุน
- Animations ยังไม่ดีพอ

การเปลี่ยนแปลงและเหตุผล (Changes from Phase 1 & 2)

- UI Update: เปลี่ยน ui ทุกหน้าใหม่หมด เนื่องจากไม่ตรงตามที่โจทย์กำหนด และ จัดให้เป้นระเบียบมากขึ้น
- เพิ่มระบบ Price Range Slider ในหน้าแรก เพื่อช่วยให้ผู้ใช้กรองสินค้าตามงบประมาณได้รวดเร็วขึ้น
- Layout: ปรับปรุงหน้า Selling ให้รองรับการอัปโหลดรูปพรีวิวหลายขนาด โดยกรอบจะยืดตามขนาดรูปจริง เพื่อป้องกันภาพเพี้ยน

เพิ่ม Multi-Dropdown ค้นหาแยกตาม 77 จังหวัด และเลือก Tag สินค้า (เช่น มือสอง, สภาพดี) ได้ทันทีจากแถบเมนู เพื่อให้ง่ายต่อการค้นหา
เพิ่มหน้า My Account ให้ผู้ขายจัดการสินค้าตัวเองได้ ทั้งการกด "จบการขาย (Sold Out)" และการลบโพสต์อย่างง่ายดาย
สร้าง Admin Dashboard ใหม่ ที่สวยงาม แสดงสถิติจำนวนสินค้าแต่ละหมวดแบบ Real-time และแอดมินควบคุมสินค้าทุกชิ้นในเว็บได้
ลบ หน้า signup ออก เพื่อบังคับให้ผู้ใช้ง่าย login ผ่าน google email เท่านั้น

กระบวนการทำงานเพิ่มเติม

1. การบริหารจัดการโปรเจกต์ (Project Management)

Agile (Micro-Sprints): แบ่งการทำงานเป็นรอบสั้นๆ เพื่ออัปเดตและปรับแก้ได้ทันที
Modular Design: แยกส่วนประกอบโค้ด (เช่น Header, Footer) เป็นชิ้นย่อย เพื่อให้ดูแลและแก้ไขง่ายขึ้นในอนาคต

2. การตรวจสอบและควบคุมคุณภาพ (Monitoring)

Automated Linting: ตรวจสอบความถูกต้องของ Syntax และ CSS ล่วงหน้าเพื่อลดบั๊ก
Visual Testing: ใช้รูปภาพ Screenshot เทียบกับหน้าจอจริง เพื่อปรับเลย์เอาต์ให้ถูกต้องเป๊ะ 100%

3. การจัดการบั๊ก (Bug Management)

Root Cause Analysis: เมื่อเจอปัญหา จะไล่ตรวจสอบไปถึงต้นตอระดับฐานข้อมูล (MongoDB) ไม่ใช่แค่แก้ที่หน้าเว็บ

Fallback Strategy: ใส่ระบบป้องกันแอปฯ พัง (เช่น ตั้งค่า Default ให้ตัวเลข, รูปสำรองเวลาภาพโหลดไม่ขึ้น) เพื่อให้ระบบรันต่อได้เสมอ

4. เครื่องมือจัดการโค้ด (Advanced Tools)

Git (Version Control): จัดการประวัติการอัปเดตไฟล์ ทำให้ย้อนกลับ (Rollback) หรือตรวจสอบโค้ดได้ง่าย

Global CSS: จัดกลุ่มสไตล์ที่ใช้บ่อยๆ ไว้ที่ส่วนกลาง เพื่อลดความซ้ำซ้อนของโค้ด

สรุปการประชุม Retrospective

Sprint 3 Summary : การประชุม Retrospective — โครงการ Global2Hand (Phase 3)

สำหรับการดำเนินงานใน Phase 3 ทีมผู้พัฒนาโครงการ Global2Hand
ได้มุ่งเน้นไปที่การพัฒนาระบบจากหน้าเว็บไซต์แบบ Static (HTML พื้นฐาน) ให้กลายเป็นเว็บแอปพลิเคชันแบบ Dynamic ที่สามารถทำงานได้จริง โดยเปลี่ยนเทคโนโลยีมาใช้ Node.js, Express Framework, EJS (Template Engine) และเชื่อมต่อกับฐานข้อมูล MongoDB ผ่าน Mongoose
เพื่อจำลองการทำงานของระบบประมวลผลข้อมูลหลังบ้าน (Backend)

ฟีเจอร์และหน้าจอหลักที่ได้รับการพัฒนาและปรับปรุงให้สมบูรณ์ในเฟสนี้ ได้แก่ ระบบค้นหาขั้นสูง
ที่รองรับการกรองตาม 77 จังหวัดและ Tags, การอัปเกรดหน้า My Account
ให้ผู้ใช้สามารถจัดการสินค้าของตัวเองได้โดยตรง เช่น การจำกัดการมองเห็นและการตั้งสถานะจบการขาย (Sold Out), โครงสร้างหน้า Wishlist, รวมถึงการสร้างหน้า Admin Dashboard เพื่อดูสถิติภาพรวมและจัดการระบบ

นอกจากนี้ ทีมงานยังได้ทำการปรับรื้อโครงสร้างของโค้ด (Refactoring) โดยจัดระเบียบภายใต้สถาปัตยกรรมแบบ MVC เลิกใช้ระบบเก่า และแยกส่วนประกอบของโค้ดที่ต้องใช้ซ้ำ เช่น Header และ Footer (Partials) ออกจากกัน รวมถึงการเคลียร์ไฟล์โปรเจกต์ที่ไม่ได้ใช้งานแล้ว เพื่อให้การทำงานมีประสิทธิภาพและง่ายต่อการบำรุงรักษา

**ในด้านการออกแบบ ทีมงานได้ทำการรื้อและออกแบบ UI ของทุกหน้าใหม่ทั้งหมด ตั้งแต่เริ่มต้น
โดยทำการเขียนและปรับปรุงโครงสร้างของ EJS และไฟล์ CSS ขึ้นใหม่ในทุกส่วนของเว็บไซต์** พร้อมกับการปรับ Layout ให้มีความสอดคล้องกันทุกหน้าแบบ Premium Design เช่น การจัดระยะห่าง (Spacing)
การเน้นความคมชัดของแถบเมนู และการใช้ Status Pill ที่มองเห็นพฤติกรรมของระบบได้อย่างชัดเจนที่สุด

สิ่งที่ทีมทำได้ดีในเฟสนี้คือ การสื่อสารและการแก้ไขปัญหาอย่างเป็นระบบ
ทีมสามารถสร้างโค้ดหน้าตาแอปพลิเคชันใหม่ทั้งหมดในระยะเวลาที่รวดเร็ว พร้อมกับการตัดสินใจตัดส่วนยิบย่อยออก (เช่น ยกเลิกระบบ Sign Up ธรรมดาแล้วไปใช้ Google Auth) ทำให้ระบบมีความเป็นเนื้อเดียวกัน
และสามารถตอบสนองกระบวนการทำงานได้รวดเร็วขึ้น

ส่วนปัญหาที่พบคือ ด้วยความซับซ้อนของการวางโครงสร้าง EJS และ CSS ควบคู่กันใหม่ทั้งหมด
ทำให้ระบบช่วงแรกมีข้อบกพร่องด้านไวยากรณ์ (Syntax Errors) หรือเลย์เอาต์เมนูเกิดการทับซ้อนกันบ้าง
รวมถึงข้อมูลเก่าของระบบที่ถูกเรียกมาแสดงไม่ได้เตรียมฟิลด์สำหรับครอบคลุมหน้าตาของ UI ใหม่
ทำให้รูปภาพและข้อความแสดงผลไม่ถูกต้องในบางกรณี

Link to Retrospective Youtube video